

Pattern Ladder — Corrected Build Plan

7 days / CPU only / \$0

This is the original build plan with every factual claim checked against the live dataset, the live APIs and current provider pricing, and corrected where it was wrong. Corrections are marked and the measurement behind each one is given, so nothing here has to be taken on trust.

The idea the plan is built on survives intact. Most of the numbers around it did not.

What it is

A student types "I keep failing problems where you shrink a window from the left" and gets back the pattern that describes it, an ordered ladder of problems that drill it from easy to hard, and a reason attached to every recommendation.

The original DSA search engine returns a flat list of twenty titles that share words with the query. This returns a study path, and it works when the student cannot name the technique they are missing — which is the entire situation they are in when they search.

The one idea

Every LeetCode problem ships with a `similar_questions` list. Across the corpus that is not metadata, it is a graph. Cluster it and you get problem families that LeetCode's own tags do not name; order a family by difficulty and you get a ladder.

This is correct, and it is the thing worth building. Everything below is about what it actually costs to make it work.

Correction 1 — The dataset does not have the shape the plan assumes

***Original:** "A free MIT-licensed dataset of 2,823 problems with full descriptions, tags, difficulty, acceptance rate, hints and the similar-questions graph."*

The dataset is real, MIT-licensed, public and ungated. But its **default configuration is the wrong file**. `data/train-00000-of-00001.parquet` — which is what `load_dataset("Alishohadaee/leetcode-problems-dataset")` returns — has exactly two columns:

column	type
<code>user_queries</code>	string
<code>expected_output</code>	string

It is an instruction-tuning set. No difficulty, no tags, no acceptance rate, and no `similar_questions`. It has 2,823 rows, which is where the plan's headline number comes from.

The structured table is only under `raw_data/leetcode_problems.json`, and it has **3,549 rows**, not 2,823, with 22 fields including everything the project needs.

Corrected corpus arithmetic:

	count
Rows in <code>raw_data/leetcode_problems.json</code>	3,549
Paid-only (statements are empty)	-716
Empty after HTML stripping	-3
Retrievable corpus	2,830

So "2,823 problems" is very nearly right by coincidence: it is the row count of a different file that happens to land within seven of the correct answer. A build that pointed at the default config would have produced a corpus with no difficulty, no tags and no graph, and the failure would have surfaced on day three rather than day one.

Correction 2 — The graph is sparse, and the plan's own fallback makes it worse

***Original:** "The graph is load-bearing, and you cannot confirm it is dense enough until day one."*

You can confirm it before writing any code. Measured on the 2,830-problem corpus:

	measured
Undirected edges	1,932
Problems with no link at all	1,074 (38.0%)
Mean degree among connected problems	2.2 (median 2, max 19)
Connected components	1,248 (largest holds 1,221)
Louvain families of ≥ 5 members	60
Share of corpus in any family	49.2%

The plan's "40+ problem families" bullet survives — 60 of them. But **half of all searches would return results with no ladder**, and the ladder is the product.

A note on a trap here: the modularity of that clustering is 0.93, which looks excellent. It is high only because the graph is shattered into 1,248 disconnected components, and modularity is trivially high on a disconnected graph. Reporting it as evidence of clustering quality would be unsound.

***Original fallback:** "build the graph from problems sharing two or more tags instead of from explicit links."*

This was tested and it is worse than the problem. Joining problems that share two or more tags produces 9 families of ≥ 5 members, sized 303, 290, 184, 180, 110, 107, 104, 50, 5. Those are not patterns; they are the tag pairs they were joined on. It rebuilds the taxonomy the project exists to go beyond, and would hollow out the central claim.

What to do instead: keep the curated links as the backbone and attach only *under-connected* problems to their nearest neighbour in dense-embedding space, at a lower edge weight, and only when the two also share a topic tag. The embeddings are already being computed for retrieval, so this costs nothing new.

	curated only	with backfill
Edges	1,932	4,395
Problems with no edge	1,074 (38.0%)	298 (10.5%)
Families (≥ 5)	60	137
Corpus inside a family	49.2%	86.3%
Largest family	63	42
Name coherence	0.790	0.845
Agreement with tags (NMI)	0.618	0.667

Coverage nearly doubles, families get *more* coherent, and agreement with the tag taxonomy stays around 0.6 — so the families remain substantially more than a tag rename. That last number rising from 0.618 to 0.667 is an honest cost of splitting more finely, and should not be glossed over.

Correction 3 — The cross-encoder is a trade, not "the largest quality jump"

Original: "A cross-encoder reranks the top 50. It is the largest single quality jump available and it is free."

Measured on a 20-query evaluation set, with the expected answer for each query fixed in advance:

configuration	hit@5	oblique queries	mean rank of hits
No reranking	0.90	1.00	1.56
Cross-encoder decides the order	0.85	0.83	1.06
Both orderings fused with RRF	0.95	1.00	1.53

Letting the cross-encoder overwrite the retrieval order **lowers** hit@5, and lowers it most on obliquely-phrased queries — the exact case the project exists to serve. It genuinely does sharpen precision, pulling the mean rank of found problems from 1.56 to 1.06. So it is a trade between precision and recall, not a free upgrade.

Published work reports the same shape: one study measured recall falling from 0.828 to 0.733 under cross-encoder reranking, and recommends fusing the two rankings rather than substituting one for the other. Doing that here scores 0.95, beating both alternatives.

The correction is one line of architecture: treat the reranker as a *third ranker* and fuse it, which the pipeline already has machinery for.

Correction 4 — Groq's free tier is 14x smaller than stated

Original: "Free tier gives 30 requests a minute and 14,400 a day, which is far more than a demo will ever use."

30 RPM and 14,400 RPD are the published limits for `meta-llama/llama-prompt-guard-2-22m` and `-86m`. Those are classifiers. They cannot emit structured JSON and cannot do this task.

Every chat model that can — `openai/gpt-oss-20b`, `gpt-oss-120b`, `qwen/qwen3.6-27b` — is on **30 RPM / 1,000 RPD / 8,000 TPM / 200,000 TPD**.

The binding constraint is not the daily request cap, it is **8,000 tokens per minute**. The plan calls one model to parse the query and a second to write an explanation per result, roughly 1,200 tokens a query, which caps throughput at about **six queries a minute**.

Measured, once a key was available. One call per query, with `reasoning_effort="low"`, costs 272 prompt + 100 completion = **372 tokens**. That gives **21.5 queries per minute** before the token limit bites, and -- the number nobody would predict from the plan -- **538 queries per day**, because 200,000 tokens per day divides to less than the 1,000-request daily cap. The daily binding constraint is tokens, not requests, and budgeting against requests over-estimates capacity by nearly 2x.

A separate trap: `gpt-oss-20b` is a *reasoning* model. It spends several hundred tokens reasoning before answering, and they count against `max_tokens`. A 200-token budget fails 100% of calls with a 400. Set it to 512, and set `reasoning_effort="low"` -- which cuts tokens by 40%, runs faster, and extracts better techniques than the default.

Correction: drop the second call. Explanations should be assembled from fields that were actually retrieved — the family, the difficulty, the acceptance rate, which retrieval arm found the result — not generated. That halves token spend, and removes the more serious problem: a model asked to justify a ranking it did not produce will write a plausible reason whether or not a real one exists, and a confident false explanation of a correct result teaches the student a relationship that is not there.

Correction 5 — Hugging Face Spaces is no longer free for this

Original: "Deploy to Hugging Face Spaces, free CPU tier."

From Hugging Face's own documentation, current:

Static Spaces are free for everyone. Gradio and Docker Spaces run on compute and require a paid plan to create: PRO for personal accounts, Team or Enterprise for organizations.

Streamlit is no longer offered as a Space SDK at all — the choices are Gradio, Docker, and static HTML. Free personal accounts get up to two Gradio Spaces on ZeroGPU, which is a GPU tier and not what a CPU app wants.

Correction: deploy to **Streamlit Community Cloud** — free, unlimited public apps, requires a public GitHub repository. The real constraint there is roughly **1 GB of memory per app**, which is a genuine design pressure: the two models are tiny, but PyTorch itself is not. If the resident set does not fit, the inference path moves to ONNX Runtime and PyTorch is dropped entirely.

Correction 6 — "Well under a second" is not achievable here

***Original:** "ms-marco-MiniLM-L-6-v2 ... reorders fifty candidates on a CPU in well under a second."*

Measured on a 12-core desktop CPU, reranking 50 candidates:

candidate text given to the reranker	time per query	top-10 agreement with full text	top-1 changed
Full statement (~1,300 chars)	2.85 s	—	—
900 characters	1.93 s	0.88	no
600 characters	1.32 s	0.86	no
400 characters	0.75 s	0.76	no
300 characters	0.55 s	0.74	yes

Sub-second is reachable only by truncating to 400 characters or fewer, which starts costing agreement, and at 300 the top result begins to move — the one thing that must not change. At 600 characters the measured end-to-end median is **1.32 s**, and that is on a 12-core machine; free hosting will be slower.

The honest claim is "about a second on a desktop CPU", not "well under a second". A LeetCode statement puts the task first and the worked examples last, so truncating the *head* of the text is principled rather than a pure speed hack.

Correction 7 — "Cold start under two seconds" is off by 10x

***Original:** "Precomputed indexes cached to disk ... Cold start under two seconds because nothing is computed at request time except the rerank."*

The reasoning is right and the number is wrong, because it accounts for the wrong cost. Measured:

stage	time
Import torch and sentence-transformers	0.64 s
Load cached corpus, BM25 index, embeddings, families	0.06 s
First query, including loading both model weights	20.06 s
Cold start to first answer	20.8 s
Warm query thereafter	1.36 s

Caching the indexes works exactly as intended -- loading all of them takes **60 milliseconds**. But cold start is dominated by reading ~90 MB of model weights off disk and initialising them, which caching the indexes does nothing about. The Dockerfile bakes the weights into the image

so this is a one-time cost per container rather than per deploy, but it is not two seconds.
The defensible claim is "the index costs nothing to load; the models cost 20 seconds once".

Correction 8 — Memory, which the plan never mentions

Not an error in the plan so much as an omission that the hosting change makes critical.
Measured resident memory:

stage	RSS
Bare Python	19 MB
After imports	73 MB
After all cached indexes	93 MB
After both models load	~700 MB

The indexes -- the part the plan worries about -- cost 20 MB. PyTorch and the two models cost 600. Against Streamlit Community Cloud's ~1 GB ceiling that fits, with roughly 320 MB of headroom before the Streamlit server's own footprint is counted. It is not comfortable, and it is the number to watch if anything larger is ever swapped in.

Correction 9 — Two smaller notes

The reranker model id. `cross-encoder/ms-marco-MiniLM-L-6-v2` (with a hyphen between L and 6) returns 404 from the Hub API. It still works, because file resolution redirects, but the canonical repository is `cross-encoder/ms-marco-MiniLM-L6-v2`. Depending on a redirect continuing to exist is an avoidable risk.

BM25 defaults. `bm25s` defaults to `k1=1.5`, not the commonly cited 1.2. Sweeping `k1` over [0.9, 1.8] and `b` over [0.3, 0.9] changed pool recall by at most 0.007 and did not change `hit@5` at all, so the defaults are fine — but that is a measured result rather than an assumption, and it locates the bottleneck: with the correct answer reaching the candidate pool on 100% of queries, every remaining error is in ranking, not retrieval.

Corrected cost table

Component	Choice	Real cost
Corpus	Alishohadaee/leetcode-problems-dataset, raw_data/config, 2,830 usable of 3,549 rows, MIT	free
Lexical	bm25s — NumPy only	free
Dense	all-MiniLM-L6-v2, 22.7M params	free
Reranker	cross-encoder/ms-marco-MiniLM-L6-v2, 22.7M params	free
Graph	networkx Louvain	free
Query understanding	Groq free tier — 30 RPM / 1,000 RPD / 8,000 TPM	free
Interface	Streamlit	free
Hosting	Streamlit Community Cloud (~1 GB RAM cap) — <i>not</i> HF Spaces	free
CI	GitHub Actions, unlimited on public repositories	free

Total running cost is genuinely \$0. Had the API been metered, one query at gpt-oss-20b list prices would cost about \$0.00005, or roughly five cents per thousand queries — so cost was never the risk. The rate limit is.

Corrected seven days

Day 1 — Data and indexes. Load from raw_data/leetcode_problems.json, not the default config. Strip HTML with a parser that inserts word boundaries at block tags and normalises ; both failures are silent and cost recall rather than raising. Build BM25 and the embeddings, cache both. Measure the graph: 1,932 edges, 38% isolated. Decide the backfill on day one, not day three.

Day 2 — Hybrid retrieval. BM25 and dense behind one interface, fused with RRF, then the cross-encoder over the top 50 — **fused with the retrieval order, not replacing it**. Build the 20-query evaluation set *now* rather than on day six; every tuning decision after this point needs it, and choices made before it exists are guesses that have to be revisited.

Day 3 — The graph. Louvain over the curated links plus the embedding backfill. Tune the resolution against ladder quality, not against family size: at the default resolution one community held queue design, heap/greedy and sliding-window problems at once, and Minimum Window Substring ended up in a family called "Queue / Design".

Day 4 — Query understanding. One Groq call, one strict schema, one fallback. The fallback matters more than the prompt — and it should be good enough to run the entire system, because that is what makes the project testable and demonstrable with no key. Budget 8,000 tokens per minute, not 14,400 requests per day.

Day 5 — The interface. Streamlit. Show the parsed reading before the results. Label whether it came from the model or the offline parser: a fallback parse is a weaker claim and should not

be presented as though it were the same thing.

Day 6 — Engineering. uv and a lockfile, Dockerfile, CI. The smoke test is a tripwire, not a benchmark — say so, and report per-phrasing results, because a blended average hides the oblique queries that are the whole point.

Day 7 — Ship. Streamlit Community Cloud. Measure resident memory against the 1 GB ceiling before claiming it is deployed.

Corrected bullets

The original bullets, with the numbers that are actually defensible.

Built a DSA study engine over 2,830 problems, fusing BM25 and dense retrieval with RRF and a CPU cross-encoder, reaching 0.95 hit@5 and 1.00 on queries that never name the technique

Discovered 137 problem families by Louvain community detection over LeetCode's similar-questions graph, backfilling a 38%-isolated graph to 86% coverage without collapsing the families into tags

Turned loose queries into structured intent with a schema-validated LLM call and a deterministic fallback that runs the whole system unaided, showing the parsed reading back so a wrong one is visible not silent

Shipped on cached indexes behind Docker and CI with 147 offline tests, at zero running cost

The one thing to know going in

The original said the graph was load-bearing and could not be checked until day one. It is load-bearing, it can be checked in ten minutes before any code is written, and **it is not dense enough on its own**: 38% of problems have no link, and clustering the raw graph leaves half the corpus with no ladder.

The fix is not the tag fallback the plan proposed — that is measurably worse, because it reconstructs the tags. It is to use the embeddings you are already computing to attach the strays, conservatively, with the curated links still deciding where the boundaries fall.